PRINTXYZ

Automated Logistics Integration

Implementation brief for the AI coding agent building the Printxyz website. Defines the intended workflow, admin controls, API behaviour, security requirements and acceptance tests.

Brand token	Exact value	Use
Deep navy	#0F172A	Main brand and text
Orange	#F97316	Primary actions and highlights
White	#FFFFFF	Main background
Soft grey	#F8FAFC	Subtle panels and sections

CORE REQUIREMENT: Do not create a courier shipment when an order is first placed. Create it only after the seller marks the item printed, quality-checked, packed, and ready to ship.

1. Goal and scope

Build a mostly automated shipping workflow for Printxyz. The seller should not need to retype customer addresses into a courier dashboard. Printxyz should send eligible order and parcel data from its backend to a shipping provider such as Shiprocket, request shipment creation, retrieve available labels and tracking details, and show shipping progress in the admin panel.

Use the provider's current official API documentation and confirm account eligibility, endpoints, required fields, serviceability, pickup rules and plan limitations before implementation. Do not assume every provider action is available for every account or shipment.

2. Intended end-to-end workflow

A. Order received

Save order, customer details, address, items, payment state and order ID in the existing database.

B. Production

For customised items, follow the preview-approval process. Print the product and complete quality checks.

C. Pack and measure

Seller packs the parcel and enters/validates its actual packed weight and dimensions. Never invent measurements.

D. Ready to Ship

Seller clicks a one-click admin action. Backend checks eligibility and requests shipment creation.

E. Courier and label

Show eligible courier options or use a configured rule. Request courier assignment and label generation using supported API endpoints.

F. Pickup

Allow pickup scheduling or confirmation as supported by the provider. Show failures and safe retry options.

G. Tracking

Sync tracking using provider webhooks where available, with safe polling fallback if needed. Show tracking link/status to customer.

H. Delivery and exceptions

Update shipment status, surface delivery exceptions, and keep order, payment, production and shipment states separate.

3. Admin interface requirements

Add a Shipping / Fulfilment panel to each eligible order. It should show:

• Order ID, customer name, delivery address and contact details (authorised admins only).

• Payment, production and fulfilment statuses as separate fields.

• Packed weight and parcel dimensions, with validation and no silent defaults.

• Courier provider, selected service, shipping charge, shipment ID, AWB/tracking number and tracking URL.

• Label download/print action when the provider returns a label.

• Pickup scheduling status, latest tracking event and shipment error.

• Clear Ready to Ship / Create Shipment action, disabled when prerequisites are unmet.

• Safe retry, tracking refresh, shipment cancellation where supported, and manual exception recording.

4. Suggested status model

Category	Example states	Rule
Payment	Pending, Partially paid, Paid, Failed, Refund pending, Refunded	Verify payment server-side; never trust a browser success redirect alone.
Production	Awaiting approval, Approved, Printing, Quality check, Packed, Ready to ship	Seller-controlled production workflow; custom orders may require approval and advance payment.
Shipment	Not booked, Booking pending, Booked, Pickup scheduled, Picked up, In transit, Delivered, Exception, Cancelled, Returned	Update from verified provider events/API responses and keep event history.

5. Backend/API implementation requirements

• Keep shipping-provider API credentials in server-side environment variables or a secret manager. Never expose credentials in frontend code or browser storage.

• Create a backend shipping service/module; keep provider-specific logic out of UI components.

• Use official current API documentation. Do not guess endpoint paths, payload fields, status names or signature formats.

• Check admin authorisation and order eligibility on the server before creating/changing a shipment.

• Validate address, phone, serviceability, parcel weight/dimensions, payment/COD rules and production readiness.

• Make shipment creation idempotent. Prevent duplicate shipments from double-clicks, retries or webhook replays. Store provider IDs and an internal shipment record.

• Handle timeouts, rate limits, provider errors, retries with backoff and partial failures. Show useful errors without exposing secrets.

• Verify webhook signatures if supported. Treat incoming events as untrusted until verified; handle duplicates and out-of-order events safely.

• Record an audit trail for shipment creation, label retrieval, pickup requests, cancellations, tracking updates and admin actions.

• Do not log credentials, payment secrets or unnecessary customer personal information.

6. Customer notifications

When enabled and supported by the existing messaging setup, notify customers after shipment booking is confirmed and provide the tracking link. Send further updates for pickup, transit, delivery or exceptions only when appropriate. Prevent duplicate messages when events are retried; keep transactional messages separate from marketing.

If notification integration is not configured, document the required credentials/setup. Never claim a message was sent without confirmation.

7. Important business rules

• Do not book a courier merely because an order was placed.

• For custom products, respect existing preview approval and payment policy before production.

• Require seller confirmation that the parcel is packed and actual weight/dimensions are entered.

• Do not mark an order paid because a shipment was booked or delivered.

• Do not mark a shipment delivered merely because an admin page was opened.

• Offer COD only if enabled for the order, business policy and shipping provider account. Track COD collection/settlement separately.

• Do not promise insurance or compensation unless confirmed for the selected service/account.

• Provide a manual fallback to save courier and tracking details if the API is unavailable.

8. Data to persist

Field group	Examples
Internal order	Order ID, customer ID, address snapshot, item list, totals, creation time
Parcel	Packed weight, length, width, height, units, packing confirmation timestamp
Provider reference	Provider, shipment ID, internal reference, courier/service, AWB/tracking ID, label URL or safe storage reference
Status/history	Current status, provider event IDs/timestamps, errors, retry count, audit events
Costs	Quoted shipping charge, final charge if available, COD amount if applicable, invoice/settlement references

9. Security and privacy

• Only authorised admins can view customer addresses or operate shipping controls.

• Protect admin endpoints with existing authentication and role checks.

• Validate all user and provider input on the backend.

• Do not expose customer address/phone details to unrelated users or public tracking APIs.

• Use HTTPS for provider callbacks and external links where required.

• Store only data needed for fulfilment and follow the site's privacy/retention policy.

10. Acceptance tests

■ An eligible packed order can create one shipment through the admin action.

■ Double-clicking or retrying does not create a duplicate shipment.

■ An order that is unpaid when payment is required, not approved, or not packed cannot be shipped accidentally.

■ Invalid address or missing parcel dimensions produce a clear error and no duplicate shipment.

■ Provider failure is visible and can be retried safely.

■ Label can be retrieved/printed when supported.

■ Tracking updates are safe to process more than once.

■ Out-of-order events do not incorrectly regress shipment state without a documented reason.

■ Unauthorised users cannot create shipments or see private customer information.

■ Manual courier booking can be recorded if the API is unavailable.

■ Existing checkout, payment verification, order history and admin functions still work.

11. Delivery plan

Stage 1 — Inspect

Review repository, order model, admin dashboard, backend stack, deployment and official shipping API documentation.

Stage 2 — Backend

Add shipment data model, secure provider service, idempotency, error handling and admin-only endpoints.

Stage 3 — Admin UI

Add parcel measurements, Ready to Ship action, shipment details, label action, pickup controls, tracking and retry UI.

Stage 4 — Tracking/notifications

Integrate verified webhooks or safe polling; connect notifications only if configuration exists.

Stage 5 — Test

Use provider test mode/sandbox if available. Do not create real paid shipments without the owner's explicit approval.

12. Instructions to the AI coding agent

Read this entire brief and inspect the existing project before changing files. First report the current stack, relevant order/admin files and any missing information. Then implement the feature in the existing architecture in small stages. Run available checks and report what is complete versus what still needs provider credentials, account setup or manual configuration. Never claim a live courier booking, label, pickup or notification unless the actual provider response confirms it.

Provider reference: Shiprocket API documentation — https://apidocs.shiprocket.in/?version=latest. Confirm current API behaviour and account eligibility directly from the provider's official documentation before implementation.